

Build Rules

AI-Based Phishing and Social Engineering Detection Assistant (PS57)

Companion to [PRD_PhishGuard_PS57.md](#) and [ARCHITECTURE_PhishGuard_PS57.md](#).

This file is the ground rules for the Phase 2 build (Aug 25–29) — for the team, and for any AI coding tool (Claude Code, Cursor, etc.) being pointed at this repo. Keep it loaded alongside the architecture doc so generated code doesn't drift from the agreed design.

1. Non-Negotiables

- **Follow the module boundaries in [ARCHITECTURE_PhishGuard_PS57.md](#).** Don't collapse heuristics, PII anonymization, and LLM calls into one giant function — they need to stay separately testable and separately explainable in the live demo.
 - **PII anonymization happens before every LLM call, no exceptions.** This is a stated privacy commitment, not optional scope. If a code change routes raw content to the LLM API, that's a bug, not a shortcut.
 - **The 3 demo test cases must always work offline.** Hardcoded/pre-computed responses for the sample-case buttons are mandatory, not a "nice to have" — a live demo that depends on network + API uptime is a live demo that can fail in front of judges.
 - **Every LLM call must handle failure.** No unhandled promise rejections, no blank screens. Timeout → fall back to heuristic-only score, per the failure modes table in the architecture doc.
 - **Risk scores are weighted combinations, never a single binary signal.** Don't let one heuristic (e.g., a keyword match) alone produce a "Phishing" verdict — that's how false positives happen on Case 3.
-

2. Scope Discipline

- **Build only what's in the architecture doc for Phase 2.** No auth, no database/persistence, no homoglyph detection, no multi-language support, no Chrome extension — those are explicitly Future Scope. Adding them now eats time you don't have.

- **4 days, 4 phases** (heuristics → LLM pipeline → UI → demo testing). Don't parallelize phases in a way that leaves nothing working end-to-end until the last day — get a thin vertical slice (one input → one score → one badge) working on Day 1 if possible, then widen it.
 - If a teammate wants to add something not in the PRD/architecture, it goes in the "Future Scope" conversation, not into the Aug 25–29 sprint.
-

3. Code Organization Rules

- Match the folder structure in the architecture doc (`/lib/heuristics/`, `/lib/llm/`, `/lib/scoringEngine.ts`, etc.). Don't scatter logic into API route files — routes should orchestrate, not implement.
 - Each heuristic sub-check (Levenshtein, anchor mismatch, TLD risk, keyword scan) lives in its own file and is independently callable/testable. If someone asks "how does X work" during judging, you should be able to point at one small file.
 - Config values (protected brands list, risky TLD list, keyword bank, scoring weights) live in dedicated files, not hardcoded inline — you will want to tune weights after running the test cases, and hunting through logic to find a magic number wastes time you don't have.
-

4. LLM Integration Rules

- System prompt must enforce strict JSON-only output — no prose, no markdown fences around the JSON. Validate the response is parseable before trusting it.
 - Use a low temperature for consistency — you want the same input to produce roughly the same verdict across demo runs.
 - Never send raw, non-anonymized content to the LLM API.
 - If the LLM response fails to parse or times out: retry once, then fall back to a generic "Suspicious — manual review recommended" verdict. Never crash or hang the UI.
-

5. Testing Discipline

- All 3 test cases (blatant phishing, sophisticated social engineering, legitimate email) must be run against the pipeline before Aug 29 — not just spot-checked once.
- Case 3 (legitimate message) is the one most likely to get a team false-flagged in front of judges. Treat a false positive on Case 3 as a launch blocker, not a minor bug.

- After running all 3 cases, revisit the 0.4/0.6 heuristic/LLM weighting — it's a starting point from the PRD, not a fixed constant. Adjust if the scores don't make sense.
-

6. Working With AI Coding Tools

- Point the tool at `ARCHITECTURE_PhishGuard_PS57.md` before asking it to generate code — don't let it invent its own module structure.
 - Review generated code against the "Non-Negotiables" in §1 above before merging, especially the PII-before-LLM-call rule — AI coding tools will happily wire the raw input straight to the API call if not told otherwise.
 - Keep generated heuristic logic (Levenshtein, TLD scoring, etc.) simple enough that a teammate can explain it line-by-line if a judge asks — don't accept an overly clever one-liner over a readable implementation.
-

7. Demo-Day Rules

- Test on the actual Wi-Fi/network conditions you expect at the venue before Aug 29, not just on a home connection.
- Whoever presents should be able to answer: "why not just use a blocklist," "what's your false-positive rate," and "how do you protect user privacy" — without checking notes. These map directly to Slides 2 and 5 and to the failure-modes discussion above.
- Have the sample-case buttons as the primary demo path. Live-typing a new example is a bonus, not the plan.